

Ordonnancement Temps Réel et Analyse d'Ordonnançabilité

BRUN Xavier 21910332, HERZBERG Dwayne 22000086

18/01/2025

Introduction

Ce rapport présente les travaux réalisés dans le cadre d'un TP consacré aux systèmes temps réel. Il est structuré en deux parties principales :

- **Partie I: Conception d'application temps réel**
Cette partie aborde la conception et la mise en œuvre d'une application temps réel avec spécification en temps réel.
- **Partie II: Ordonnancement Temps Réel et Analyse d'Ordonnançabilité**
Cette partie explore les algorithmes d'ordonnancement en temps réel, les analyses d'ordonnançabilité et les résultats obtenus lors des expérimentations.

Partie I: Conception d'application temps réel

Étape 1 :MSC

Les diagrammes ci-dessous illustrent plusieurs MSC représentant le système de l'usine. Nous avons imaginé une spécification où le superviseur organise et contrôle toutes les machines et robots du système. Les pièces ne sont pas représentées par un processus, ni le convoyeur.

Dans les différents diagrammes, l'objectif principal est que le processus superviseur envoie les signaux aux moments appropriés aux différentes machines. Il conserve en mémoire l'état de chaque machine et du convoyeur, ce qui lui permet d'envoyer les bons signaux, même dans des cas complexes nécessitant la gestion de plusieurs pièces simultanément.

Le système comporte plusieurs processus :

- **pSupervisor** : Superviseur qui contrôle l'ensemble du système. Tous les autres processus communiquent avec lui.
- **pMachine** : Liste de processus machines permettant d'appliquer un usinage spécifique (percer, souder, etc.), représentés par un entier dans le système.
- **pRobotAlim** : Robot d'alimentation chargé de déposer les nouvelles pièces sur le convoyeur.
- **pRobotRetrait** : Robot de retrait qui enlève les pièces usinées du convoyeur.

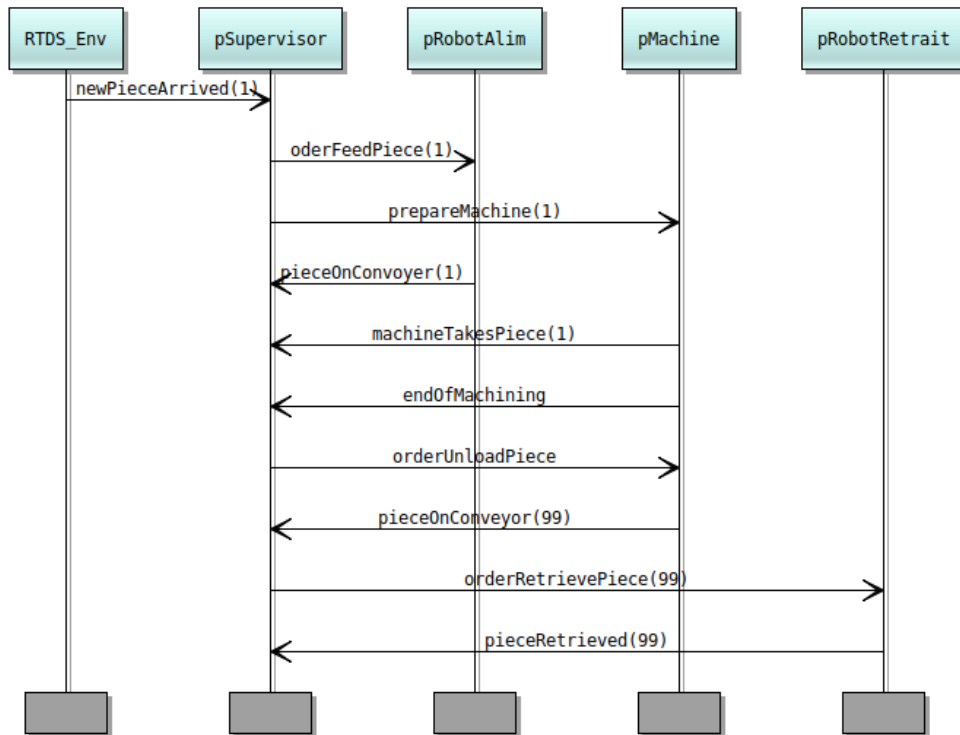


Figure 1: Diagramme MSC illustrant lorsqu'il y a qu'une seule pièce sur le système.

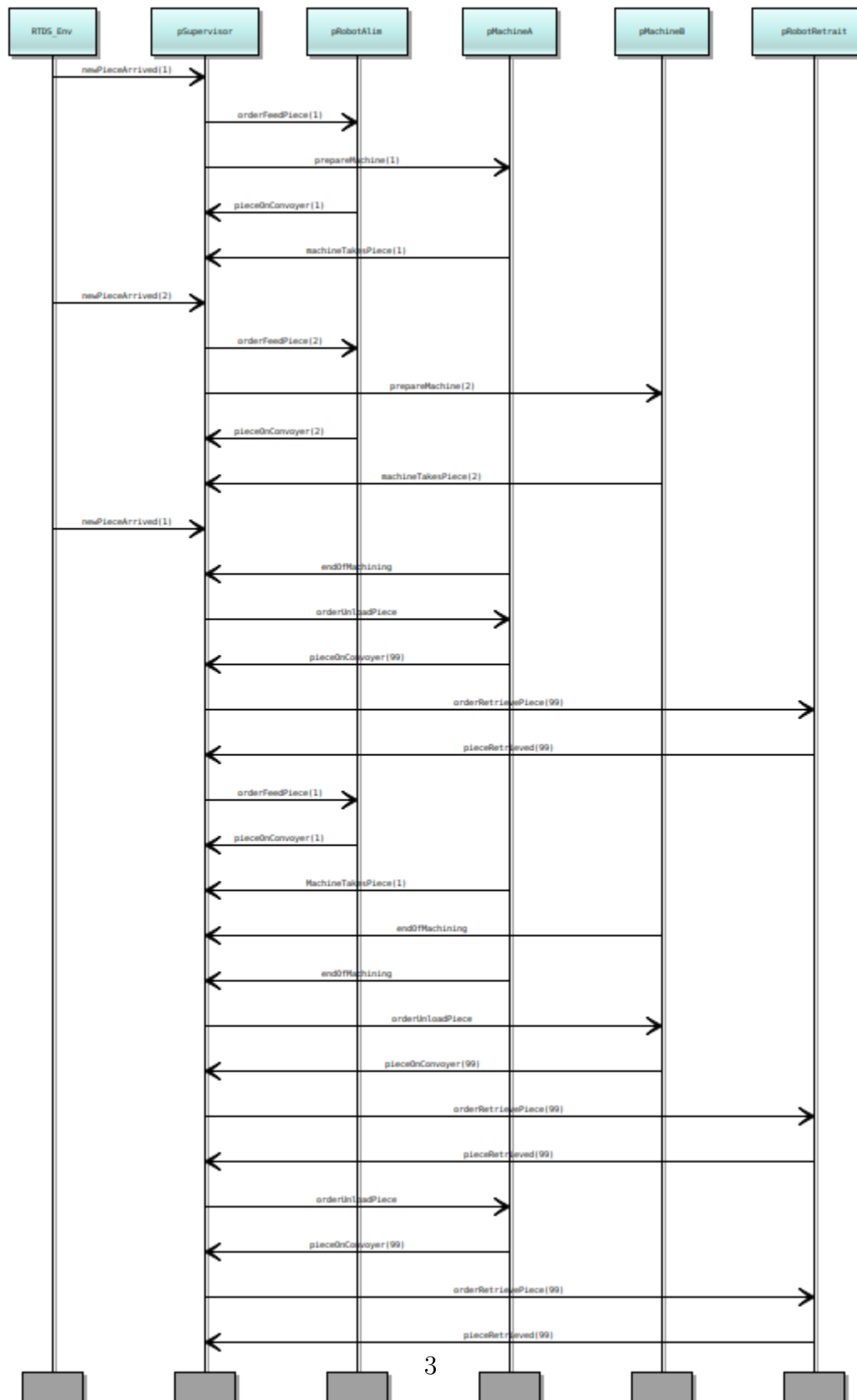


Figure 2: Diagramme MSC illustrant lorsqu'il y a plusieurs pièces sur le système avec plusieurs machines aussi.

Étape 2 :Design du système

Le système est designé de la sorte : nous avons les différents signaux qui permettent les communications entre les processus.

- **newPieceArrived(PieceNumberType)**
 - *newPieceArrived* : Ce signal est envoyé lorsqu’une nouvelle pièce arrive dans le système. Il permet au superviseur de prendre connaissance de cette nouvelle pièce.
- **orderFeedPiece, pieceOnConveyor(PieceNumberType)**
 - *orderFeedPiece* : Commande envoyée au robot d’alimentation (pRobotAlim) pour charger la pièce sur le convoyeur.
 - *pieceOnConveyor* : Indique que la pièce est maintenant sur le convoyeur.
- **prepareMachine(PieceNumberType), machineTakesPiece(PieceNumberType)**
 - *prepareMachine* : Ordre envoyé à la machine (pMachine) pour se préparer à traiter une pièce spécifique.
 - *machineTakesPiece* : Signale que la machine a pris la pièce pour son traitement.
- **orderRetrievePiece, pieceRetrieved**
 - *orderRetrievePiece* : Commande envoyée au robot de retrait (pRobotRetrait) pour récupérer une pièce finie.
 - *pieceRetrieved* : Indique que la pièce a été retirée avec succès.
- **endOfMachining(PieceNumberType), orderUnloadPiece(PieceNumberType)**
 - *endOfMachining* : Signale que le processus de traitement ou d’usinage de la pièce est terminé.
 - *orderUnloadPiece* : Commande au robot d’alimentation pour décharger la pièce traitée.
- **sendId(PieceNumberType), getId**
 - *sendId* : Signal envoyé pour transmettre l’identifiant d’une machine au superviseur.
 - *getId* : Signal indiquant que l’identifiant d’une machine est requis ou demandé.



Figure 3: Déclaration du système.

Comme dit précédemment, l'architecture choisie pour le système est que le processus **Supervisor** ordonnance le reste de tous les processus dans le système.

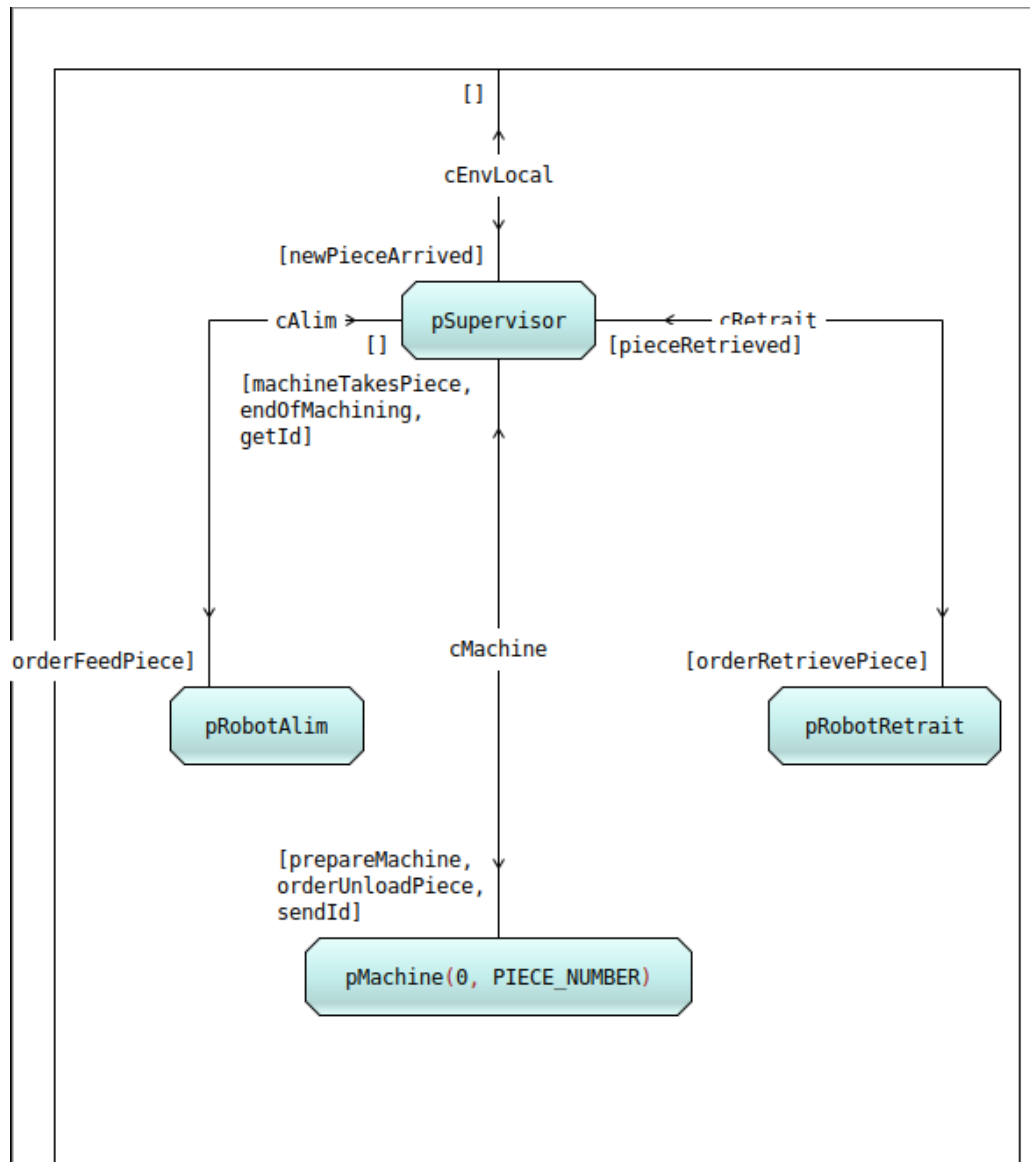


Figure 4: Bloc du système.

Étape 3 : Processus pSupervisor

L'initialisation du processus pSupervisor commence par l'appel des différents processus pMachines, correspondant aux nombres de pièces/machines (les deux sont liés) attendus dans le système (correspond à la valeur de PIECE_NUMBER).

La liste pMachines correspond aux états de chaque machine, c'est-à-dire :

- **False** : La machine est libre.
- **True** : La machine est occupée par une pièce.

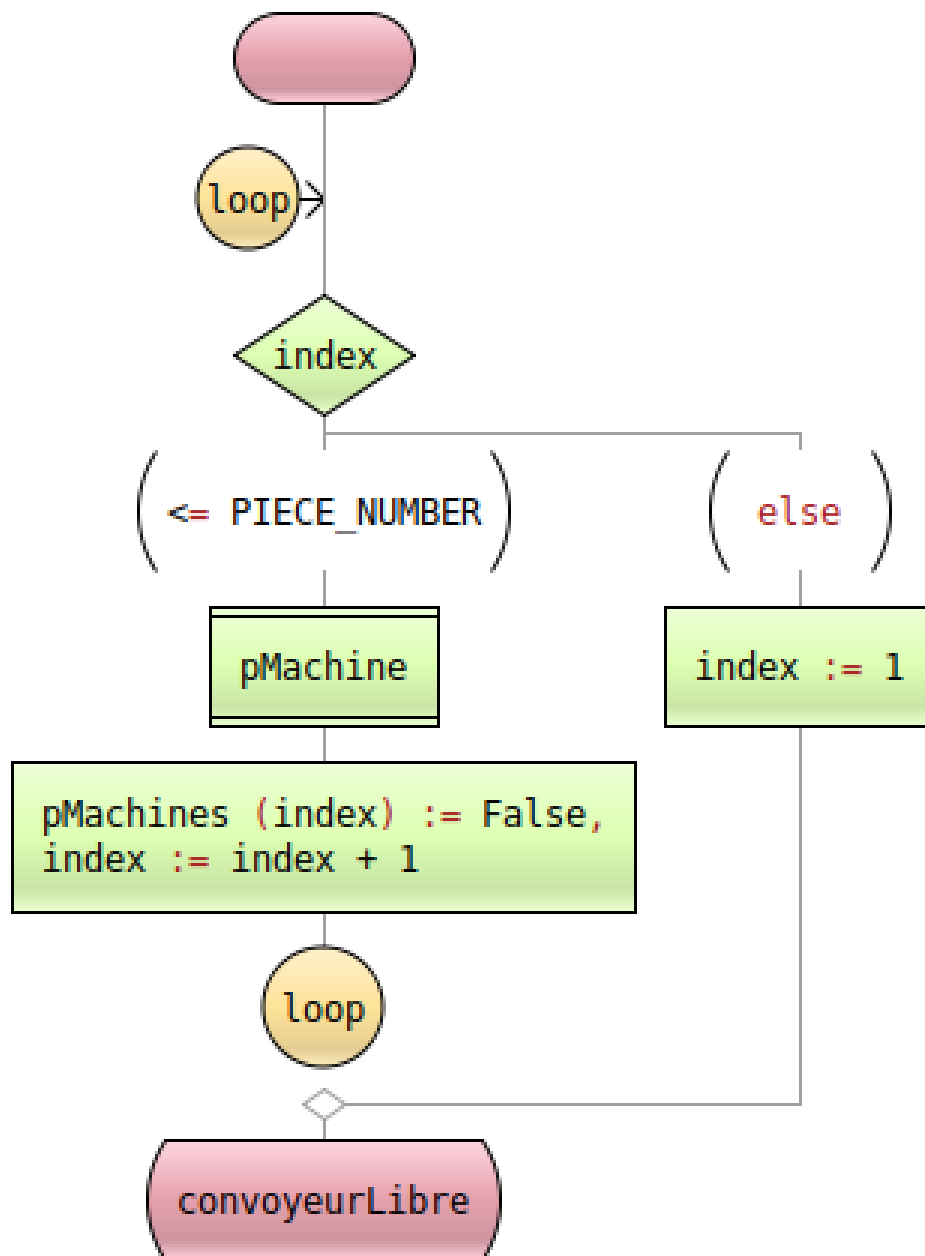


Figure 5: Organigramme pSupervisor: Initialisation des processus pMachines.

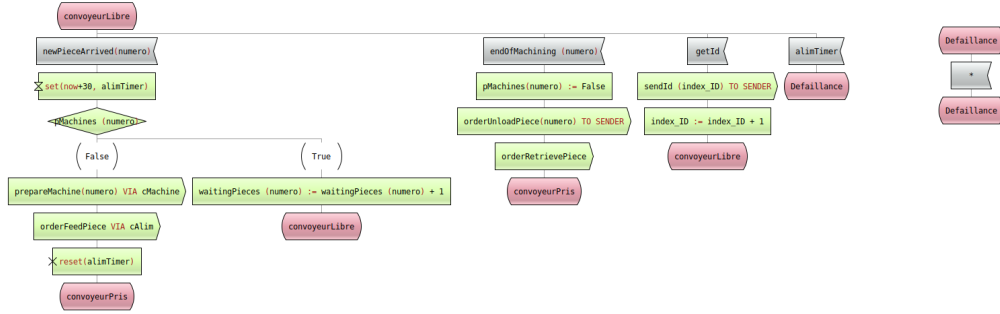


Figure 6: Organigramme pSupervisor: État du convoyeur libre.

Dans cet état, si le convoyeur est libre et qu'une nouvelle pièce arrive de l'environnement (**newPieceArrived()**), le processus effectue les étapes suivantes :

- Il lance un timer.
- Il vérifie si la machine associée à la pièce est déjà occupée (à l'aide de la liste d'état de chaque machine).
 - Si la machine est occupée, alors la pièce est ajoutée à la liste d'attente.
 - Sinon, on passe dans l'état **convoyeurPris**.

Si une machine souhaite déposer une pièce, le processus permet à cette machine de le faire uniquement si le convoyeur est libre.

Le processus gère également les identifiants (ID) de chaque machine. Chaque machine se voit attribuer un ID unique afin de permettre la communication avec une pièce spécifique dans le système.

Si les conditions de temps réel ne sont pas respectées, le processus **Supervisor** reste bloqué dans l'état **Défaillance**.

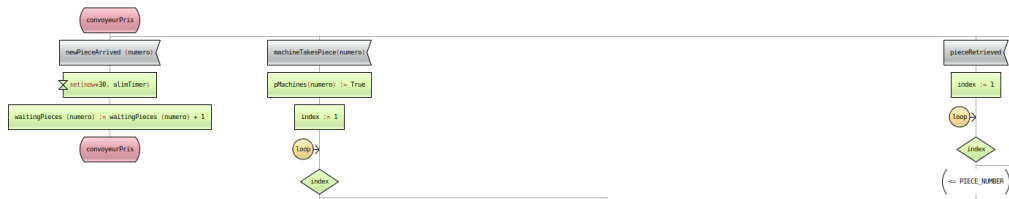


Figure 7: Organigramme pSupervisor: État du convoyeur occupé 1/2.

Dans le cas où le convoyeur est pris, le processus **Supervisor** se comporte de la même manière que lorsque le convoyeur n'est pas pris pour les cas suivants :

- Lorsqu'une nouvelle pièce arrive.
- Lorsqu'une machine souhaite déposer une pièce.
- Lorsqu'il y a une demande d'ID.
- Pour la gestion des timers.

La gestion est différente dans les cas suivants :

- Lorsqu'une machine prend une pièce.
- Lorsqu'une pièce est retirée du système par le robot de retrait de pièces.

Pour gérer ces situations, le système vérifie les listes contenant le nombre de pièces en attente dans le système :

- Si une pièce est en attente, alors cette pièce est déposée sur le convoyeur.
- Sinon, le convoyeur redevient libre.

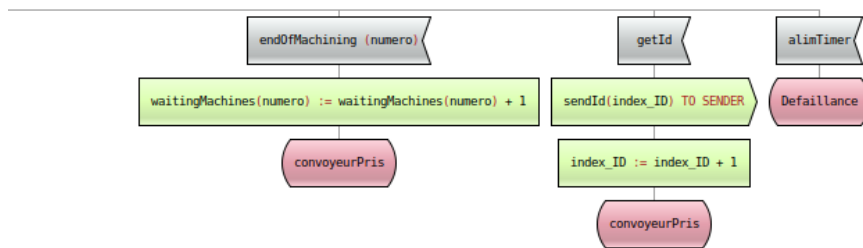


Figure 8: Organigramme pSupervisor: État du convoyeur occupé 2/2.

Étape 4 : Processus pMachine, RobotAlim et RobotRetrait

Lors de leur initialisation, les processus machines envoient une demande d'ID au processus **Supervisor**.

Les processus machines intègrent un timer qui attend pendant 20 secondes l'arrivée d'une pièce lorsqu'elle est demandée.

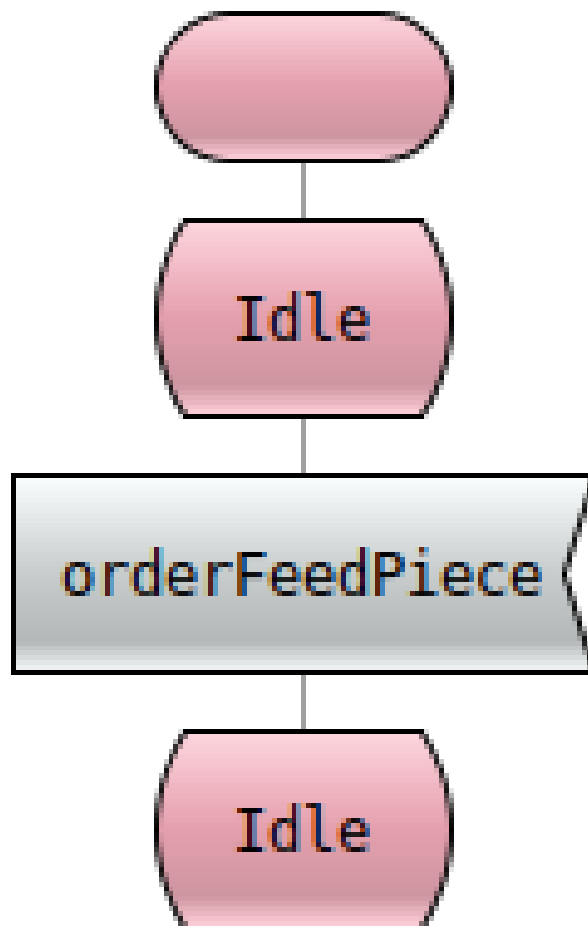


Figure 11: Organigramme pRobotAlim.

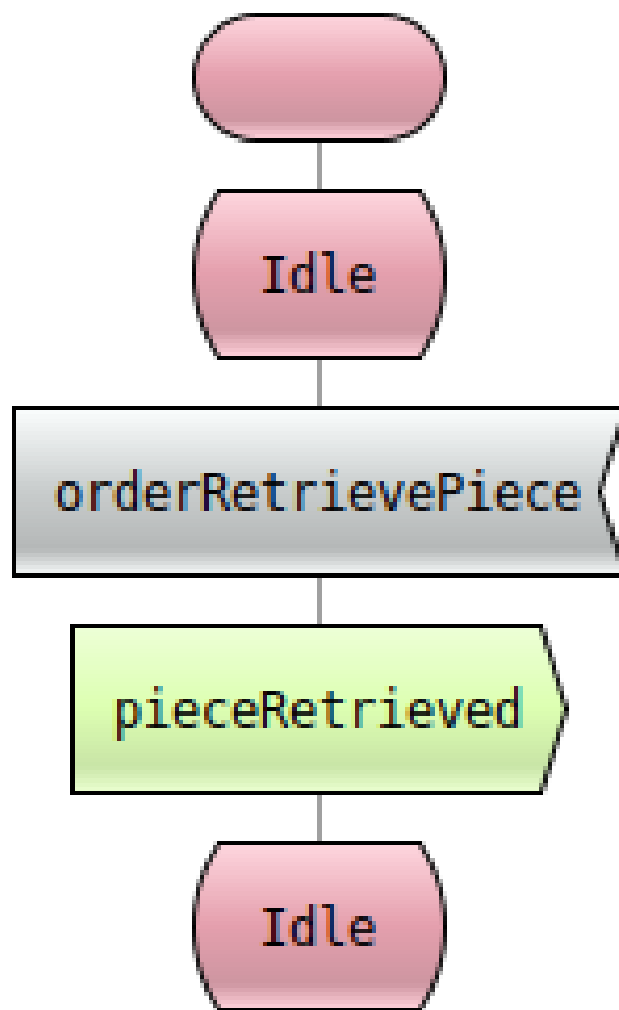
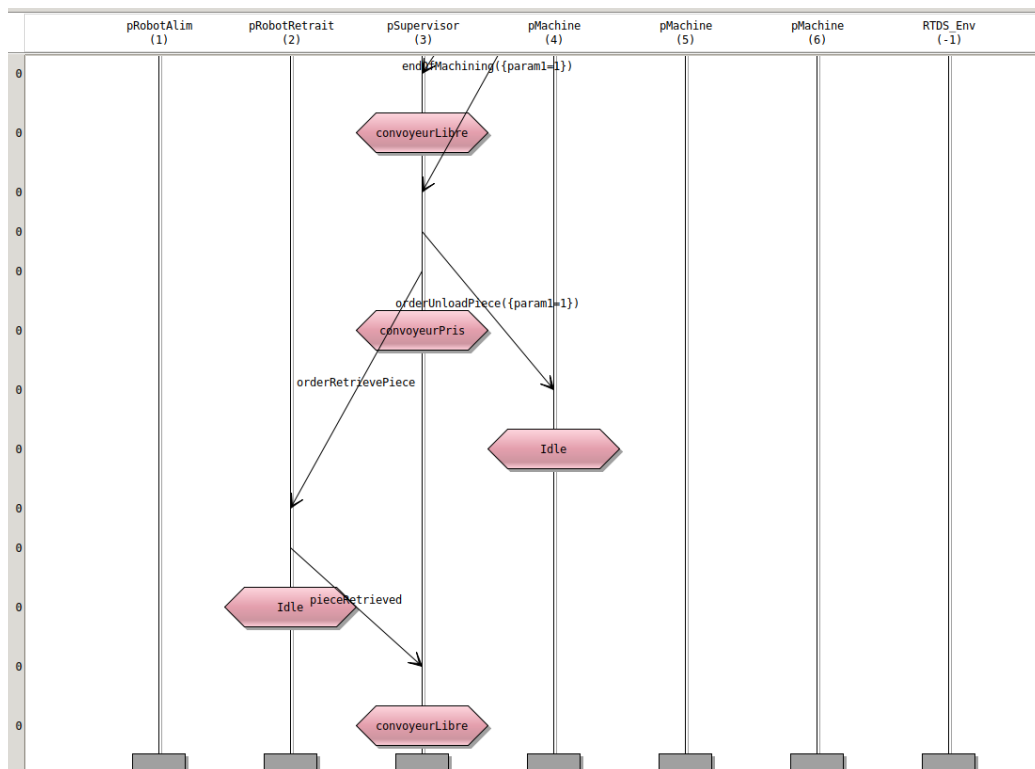
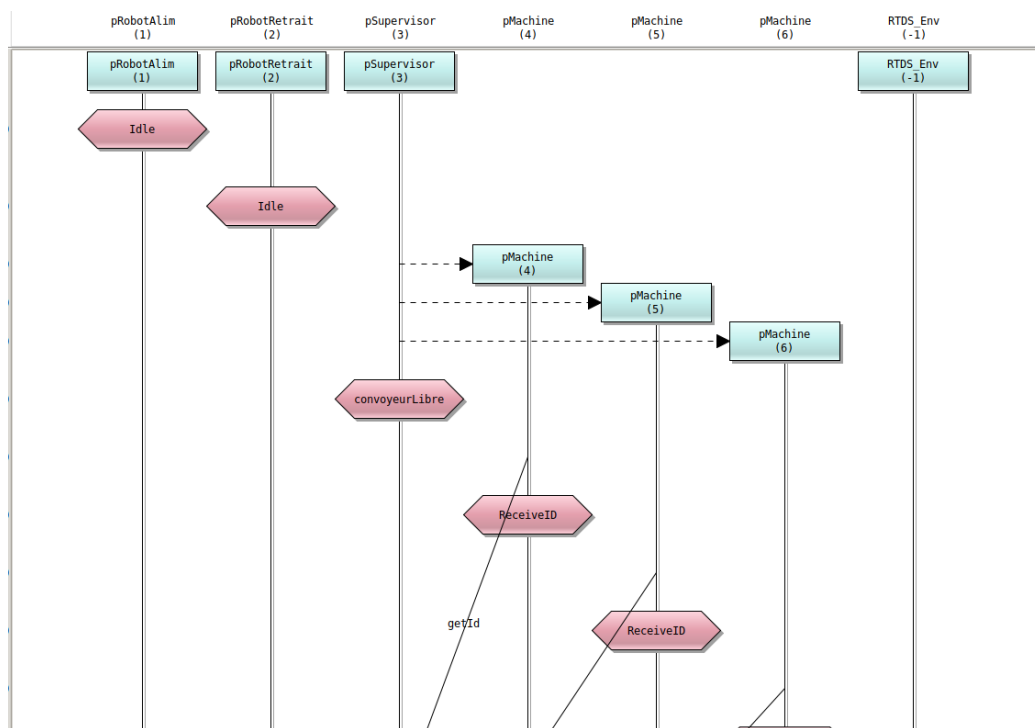


Figure 12: Organigramme pRobotRetrait.

Étape 5 : Test exécution du système

L'exécution du programme fonctionne correctement avec une seule pièce, mais rencontre des problèmes lorsqu'il s'agit de gérer plusieurs pièces avec parallélisation.

Une amélioration possible serait de mettre en place des processus spécifiques pour chaque nouvelle pièce, afin de mieux gérer la parallélisation et le traitement simultané des pièces.



1 Partie II: Ordonnancement Temps Réel et Analyse d'Ordonnançabilité

Étape 1: Étude d'algorithmes d'ordonnancement de tâches en temps réel

Les schémas présentés à l'étape 1 ont tous été générés à l'aide de scripts Python que nous avons conçus selon notre compréhension des algorithmes. L'implémentation a été réalisée par nos soins et peut comporter certaines différences par rapport à celle effectuée dans Cheddar.

Mode 1: Tâches périodiques indépendantes

Exemple 1

Tâches:

- Tâche 1: $P_1 = 8, C_1 = 2$
- Tâche 2: $P_2 = 10, C_2 = 2$
- Tâche 3: $P_3 = 8, C_3 = 1$
- Tâche 4: $P_4 = 16, C_4 = 2$

Rate Monotonic (RM) L'utilisation (U) est donnée par la formule suivante :

$$U_i = \frac{C_i}{P_i}$$

Les calculs pour chaque tâche sont les suivants :

$$U_1 = \frac{2}{8} = 0.25, \quad U_2 = \frac{2}{10} = 0.20, \quad U_3 = \frac{1}{8} = 0.12, \quad U_4 = \frac{2}{16} = 0.12$$

La somme des utilisations est :

$$U_{\text{total}} = U_1 + U_2 + U_3 + U_4 = 0.25 + 0.20 + 0.12 + 0.12 = 0.70$$

Le bound pour 4 tâches est donné par :

$$U_{\text{bound}} = n \cdot (2^{1/n} - 1), \quad n = 4$$

$$U_{\text{bound}} = 4 \cdot (2^{1/4} - 1) \approx 0.76$$

Comme $U_{\text{total}} = 0.70 < U_{\text{bound}} = 0.76$, les tâches sont **schedulables** avec RM.

Earliest Deadline First (EDF) Pour EDF, la contrainte est :

$$U_{\text{total}} \leq 1$$

Ici, $U_{\text{total}} = 0.70 \leq 1$. Les tâches sont donc **schedulables** avec EDF.

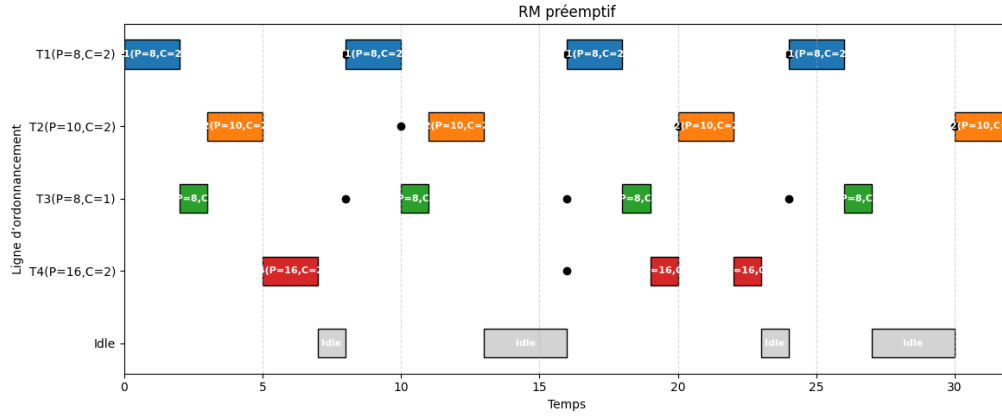


Figure 13: RM Ensemble 1

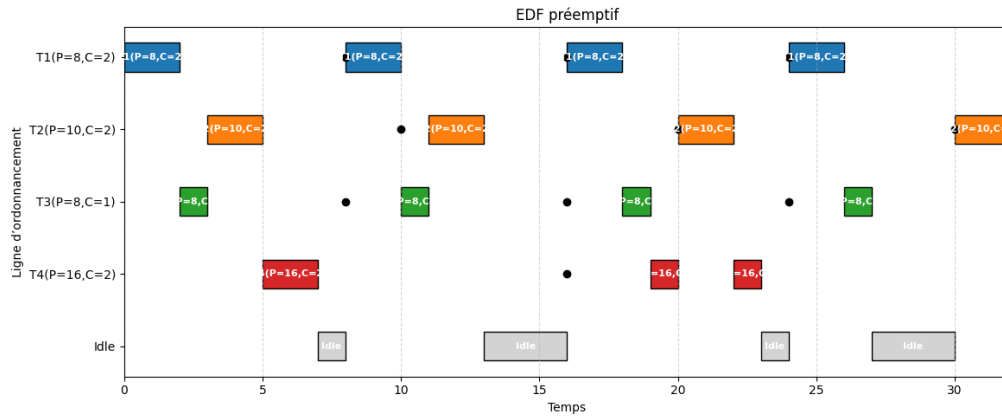


Figure 14: EDF Ensemble 1

Exemple 2

Tâches:

- Tâche 1: $P_1 = 8, C_1 = 3$
- Tâche 2: $P_2 = 16, C_2 = 3$
- Tâche 3: $P_3 = 4, C_3 = 1$
- Tâche 4: $P_4 = 8, C_4 = 1$

Rate Monotonic (RM)

$$U_1 = \frac{3}{8} = 0.375, \quad U_2 = \frac{3}{16} = 0.1875, \quad U_3 = \frac{1}{4} = 0.25, \quad U_4 = \frac{1}{8} = 0.125$$

La somme des utilisations est :

$$U_{\text{total}} = 0.375 + 0.1875 + 0.25 + 0.125 = 0.9375$$

Le bound RM est :

$$U_{\text{bound}} = 4 \cdot (2^{1/4} - 1) \approx 0.76$$

Comme $U_{\text{total}} = 0.9375 > U_{\text{bound}} = 0.76$, nous devons vérifier la condition suffisante d'ordonnancement suivante pour chaque tâche.

$$\frac{\sum_{j=1}^i \left(C_j \cdot \lceil m \cdot \frac{P_k}{P_j} \rceil \right)}{m \cdot P_k} \leq 1$$

i=1, (k=1, m=1)

$$\frac{\sum_{j=1}^1 \left(C_j \cdot \lceil 1 \cdot \frac{8}{P_j} \rceil \right)}{1 \cdot 8} = 0.3750 \leq 1$$

i=2, (k=1, m=1)

$$\frac{\sum_{j=1}^2 \left(C_j \cdot \lceil 1 \cdot \frac{8}{P_j} \rceil \right)}{1 \cdot 8} = 0.7500 \leq 1$$

i=3, (k=1, m=1)

$$\frac{\sum_{j=1}^3 \left(C_j \cdot \lceil 1 \cdot \frac{8}{P_j} \rceil \right)}{1 \cdot 8} = 1.0000 \leq 1$$

i=4, (k=1, m=1)

$$\frac{\sum_{j=1}^4 \left(C_j \cdot \lceil 1 \cdot \frac{8}{P_j} \rceil \right)}{1 \cdot 8} = 1.1250$$

i=4, (k=2, m=1)

$$\frac{\sum_{j=1}^4 \left(C_j \cdot \lceil 1 \cdot \frac{16}{P_j} \rceil \right)}{1 \cdot 16} = 0.9375 \leq 1$$

les tâches sont **schedulables** avec RM.

Earliest Deadline First (EDF)

$$U_{\text{total}} = 0.9375 \leq 1$$

Les tâches sont donc **schedulables** avec EDF.

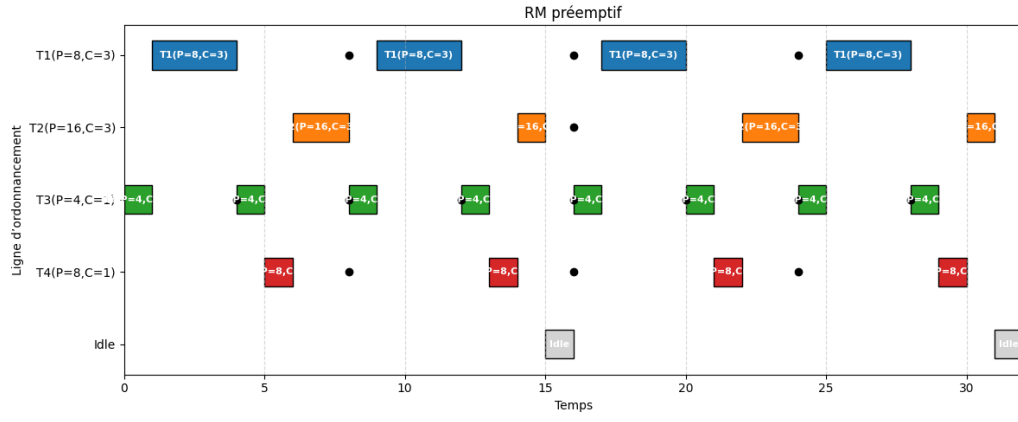


Figure 15: RM Ensemble 2

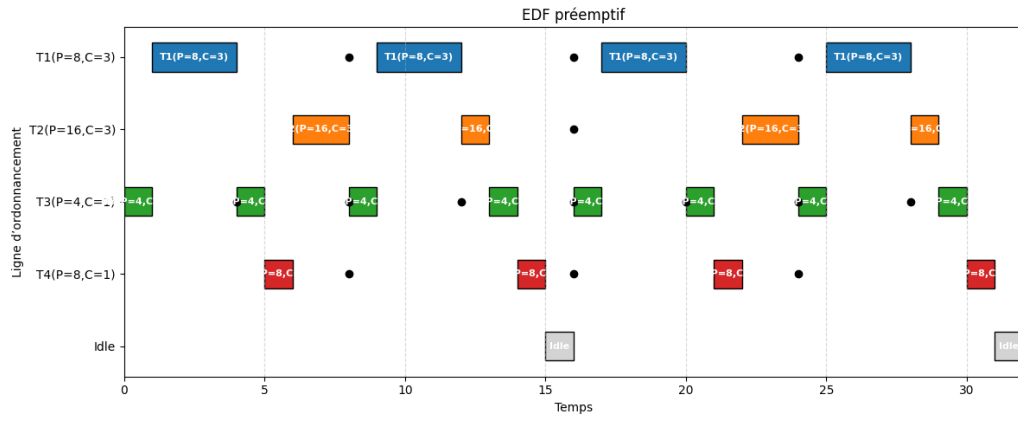


Figure 16: EDF Ensemble 2

Exemple 3

Tâches:

- Tâche 1: $P_1 = 6, C_1 = 2$
- Tâche 2: $P_2 = 9, C_2 = 3$
- Tâche 3: $P_3 = 15, C_3 = 3$
- Tâche 4: $P_4 = 8, C_4 = 2$

Rate Monotonic (RM)

$$U_1 = \frac{2}{6} \approx 0.333, \quad U_2 = \frac{3}{9} \approx 0.333, \quad U_3 = \frac{3}{15} = 0.2, \quad U_4 = \frac{2}{8} = 0.25$$

La somme des utilisations est :

$$U_{\text{total}} = 0.333 + 0.333 + 0.2 + 0.25 = 1.116$$

Le bound RM est :

$$U_{\text{bound}} = 4 \cdot (2^{1/4} - 1) \approx 0.76$$

Comme $U_{\text{total}} = 1.116 > U_{\text{bound}} = 0.76$.

i=1, (k=1, m=1)

$$\frac{\sum_{j=1}^1 \left(C_j \cdot \lceil 1 \cdot \frac{6}{P_j} \rceil \right)}{1 \cdot 6} = 0.3333 \leq 1$$

i=2, (k=1, m=1)

$$\frac{\sum_{j=1}^2 \left(C_j \cdot \lceil 1 \cdot \frac{6}{P_j} \rceil \right)}{1 \cdot 6} = 0.8333 \leq 1$$

i=3, (k=1, m=1)

$$\frac{\sum_{j=1}^3 \left(C_j \cdot \lceil 1 \cdot \frac{6}{P_j} \rceil \right)}{1 \cdot 6} = 1.3333$$

i=3, (k=2, m=1)

$$\frac{\sum_{j=1}^3 \left(C_j \cdot \lceil 1 \cdot \frac{9}{P_j} \rceil \right)}{1 \cdot 9} = 1.1111$$

i=3, (k=3, m=1)

$$\frac{\sum_{j=1}^3 \left(C_j \cdot \lceil 1 \cdot \frac{15}{P_j} \rceil \right)}{1 \cdot 15} = 1.0000$$

i=4, (k=1, m=1)

$$\frac{\sum_{j=1}^4 \left(C_j \cdot \lceil 1 \cdot \frac{6}{P_j} \rceil \right)}{1 \cdot 6} = 1.6667$$

i=4, (k=2, m=1)

$$\frac{\sum_{j=1}^4 \left(C_j \cdot \lceil 1 \cdot \frac{9}{P_j} \rceil \right)}{1 \cdot 9} = 1.5556$$

i=4, (k=3, m=1)

$$\frac{\sum_{j=1}^4 \left(C_j \cdot \lceil 1 \cdot \frac{15}{P_j} \rceil \right)}{1 \cdot 15} = 1.2667$$

i=4, (k=4, m=1)

$$\frac{\sum_{j=1}^4 \left(C_j \cdot \lceil 1 \cdot \frac{8}{P_j} \rceil \right)}{1 \cdot 8} = 1.5000$$

Les tâches ne sont donc **pas schedulables** avec RM.

Earliest Deadline First (EDF)

$$U_{\text{total}} = 1.116 > 1$$

Les tâches ne sont donc **pas schedulables** avec EDF.

Exemple 4

Tâches:

- Tâche 1: $P_1 = 6, C_1 = 2$
- Tâche 2: $P_2 = 12, C_2 = 4$
- Tâche 3: $P_3 = 24, C_3 = 4$
- Tâche 4: $P_4 = 6, C_4 = 1$

Rate Monotonic (RM)

$$U_1 = \frac{2}{6} \approx 0.333, \quad U_2 = \frac{4}{12} \approx 0.333, \quad U_3 = \frac{4}{24} \approx 0.167, \quad U_4 = \frac{1}{6} \approx 0.167$$

La somme des utilisations est :

$$U_{\text{total}} = 0.333 + 0.333 + 0.167 + 0.167 = 1.0$$

Le bound RM est :

$$U_{\text{bound}} = 4 \cdot (2^{1/4} - 1) \approx 0.76$$

$$U_{\text{total}} = 1.0 > U_{\text{bound}} = 0.76.$$

i=1, (k=1, m=1)

$$\frac{\sum_{j=1}^1 \left(C_j \cdot \lceil 1 \cdot \frac{6}{P_j} \rceil \right)}{1 \cdot 6} = 0.3333 \leq 1$$

i=2, (k=1, m=1)

$$\frac{\sum_{j=1}^2 \left(C_j \cdot \lceil 1 \cdot \frac{6}{P_j} \rceil \right)}{1 \cdot 6} = 1.0000 \leq 1$$

i=3, (k=1, m=1)

$$\frac{\sum_{j=1}^3 \left(C_j \cdot \lceil 1 \cdot \frac{12}{P_j} \rceil \right)}{1 \cdot 12} = 1.0000 \leq 1$$

i=4, (k=1, m=1)

$$\frac{\sum_{j=1}^4 \left(C_j \cdot \lceil 1 \cdot \frac{6}{P_j} \rceil \right)}{1 \cdot 6} = 1.8333$$

i=4, (k=2, m=1)

$$\frac{\sum_{j=1}^4 \left(C_j \cdot \lceil 1 \cdot \frac{12}{P_j} \rceil \right)}{1 \cdot 12} = 1.1667$$

i=4, (k=3, m=1)

$$\frac{\sum_{j=1}^4 \left(C_j \cdot \lceil 1 \cdot \frac{24}{P_j} \rceil \right)}{1 \cdot 24} = 1.0000 \leq 1$$

Les tâches sont donc **schedulables** avec RM.

Earliest Deadline First (EDF)

$$U_{\text{total}} = 1.0 \leq 1$$

Les tâches sont donc **schedulables** avec EDF.

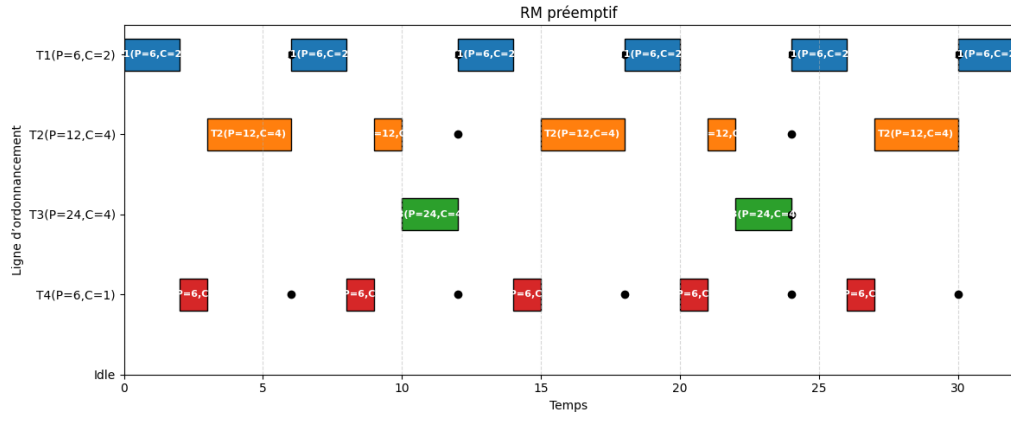


Figure 17: RM Ensemble 4

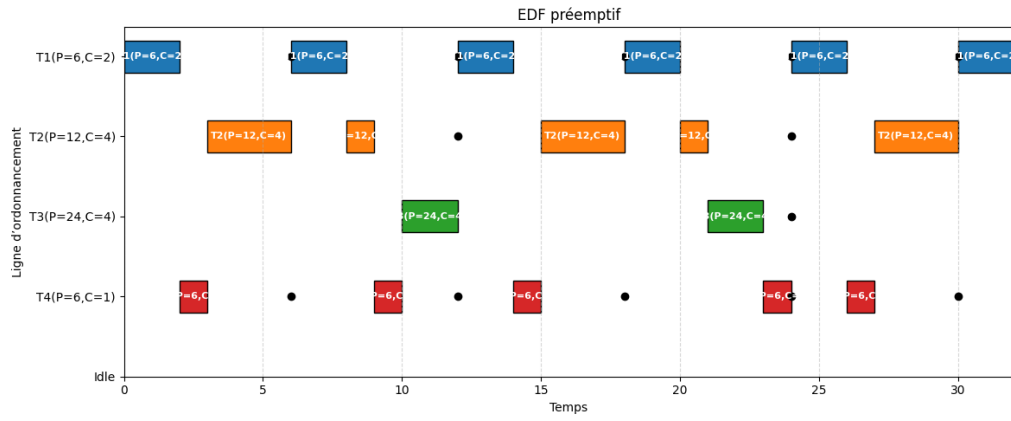


Figure 18: EDF Ensemble 4

Mode 2: Tâches périodiques et tâches apériodiques indépendantes

Pour évaluer l'ordonnabilité des tâches périodiques et du serveur de scrutation avec l'algorithme RM, nous calculons le taux d'utilisation total U_{total} , défini comme la somme des taux d'utilisation des tâches périodiques et du serveur.

Le taux d'utilisation d'une tâche périodique T_i est donné par $U_i = \frac{C_i}{P_i}$, où C_i est la capacité et P_i la période. Pour les tâches données :

$$U_1 = \frac{2}{8} = 0.25, \quad U_2 = \frac{2}{16} = 0.125, \quad U_3 = \frac{1}{8} = 0.125, \quad U_4 = \frac{1}{8} = 0.125$$

Le taux d'utilisation total des tâches périodiques est donc :

$$U_{\text{périodiques}} = U_1 + U_2 + U_3 + U_4 = 0.25 + 0.125 + 0.125 + 0.125 = 0.625$$

Le serveur de scrutation a un taux d'utilisation donné par $U_{\text{serveur}} = \frac{C_{\text{serveur}}}{P_{\text{serveur}}}$. Avec $C_{\text{serveur}} = 4$ et $P_{\text{serveur}} = 16$, on a :

$$U_{\text{serveur}} = \frac{4}{16} = 0.25$$

Le taux d'utilisation total est la somme des deux :

$$U_{\text{total}} = U_{\text{périodiques}} + U_{\text{serveur}} = 0.625 + 0.25 = 0.875$$

Comme $U_{\text{total}} \leq 1$, le système est ordonnable avec un serveur de scrutation sous l'algorithme Rate Monotonic (RM).

Ci-dessous, les tâches périodiques ont été planifiées avec l'algorithme Rate Monotonic (RM), tandis que deux tâches aperiodiques ont été gérées par un serveur de scrutation avec une période de 16 et une capacité de 4. Le diagramme Gantt illustre la séquence des exécutions périodiques et des arrivées aperiodiques.

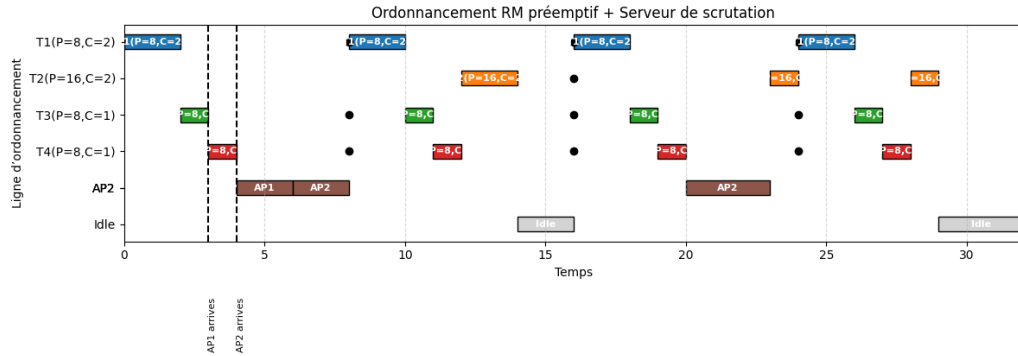


Figure 19: RM, Scrutation, P=16 C=4

Le diagramme ci-dessous illustre le même contexte, mais avec un serveur de scrutation ayant une période de 6 et une capacité de 2.

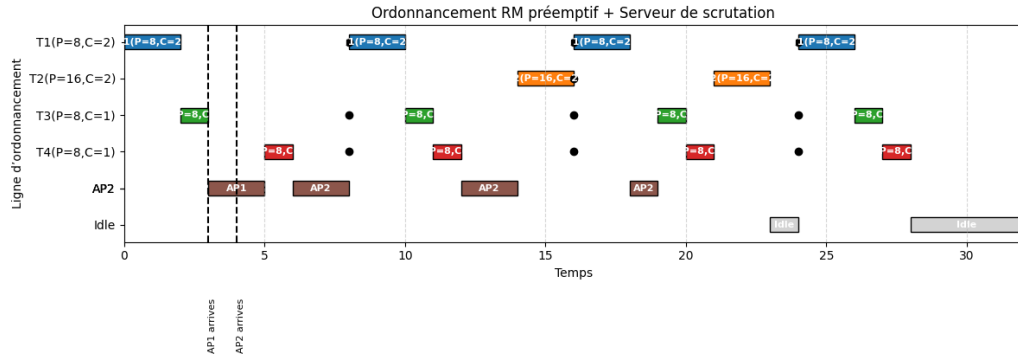


Figure 20: RM, Scrutation, P=6 C=2

Le calcul du taux d'utilisation est identique pour un serveur de scrutation et un serveur sporadique car, dans les deux cas, le taux d'utilisation est uniquement fonction de la capacité C_{serveur} et de la période P_{serveur} du serveur. La différence entre les deux mécanismes réside dans leur mode d'exécution.

Le diagramme ci-dessous illustre le même contexte, mais en utilisant un serveur sporadique au lieu d'un serveur de scrutation. La période et la capacité du serveur sporadique sont identiques : 16 et 4 dans le premier cas, 6 et 2 dans le second.

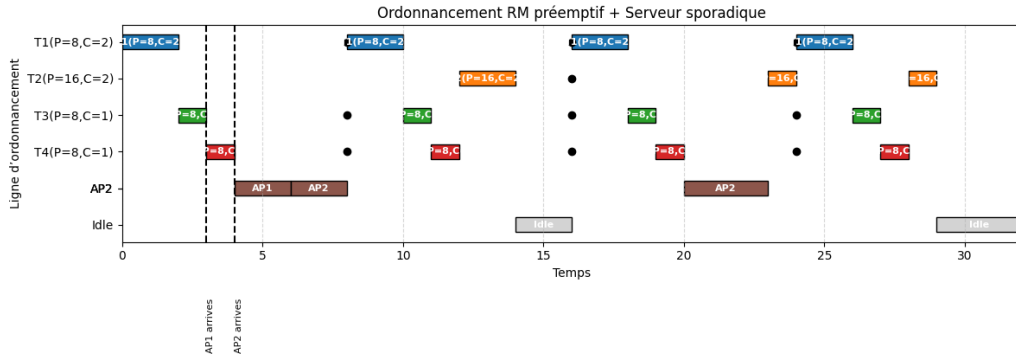


Figure 21: RM, Sporadique, P=16 C=4

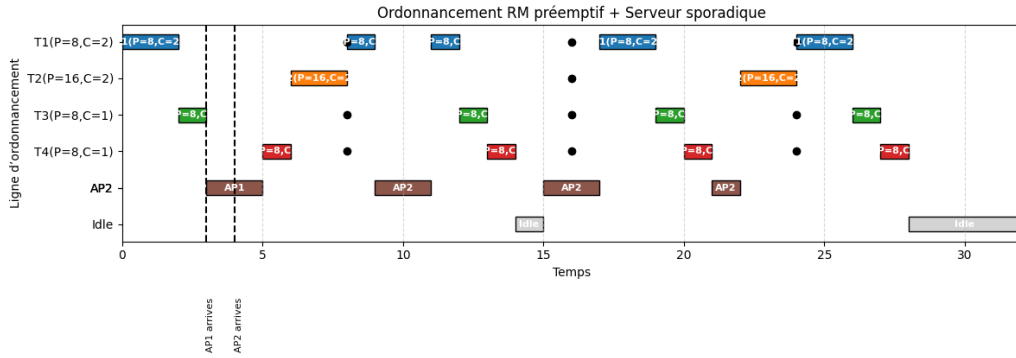


Figure 22: RM, Sporadique, P=6 C=2

Le Slack-Stealing implémenté ici repose sur l'utilisation des temps libres (slack) laissés par les tâches périodiques dans un système d'ordonnancement EDF sans préemption. À chaque instant, si aucune tâche périodique prête n'est en attente d'exécution (c'est-à-dire que toutes respectent leurs deadlines), les tâches apériodiques peuvent utiliser ce temps disponible pour s'exécuter. Ces tâches apériodiques sont servies selon une politique FIFO (première arrivée, première servie). Le processus vérifie en permanence l'état des tâches périodiques : dès qu'une tâche périodique devient prête, elle est exécutée en entier selon sa priorité EDF (plus la deadline est proche, plus la tâche est prioritaire).

Pour cette méthode, le taux d'utilisation des tâches périodiques $U_{\text{périodiques}}$ est calculé comme suit :

$$U_{\text{périodiques}} = \frac{2}{8} + \frac{2}{16} + \frac{1}{8} + \frac{1}{8} = 0.625$$

Le slack, ou temps inutilisé du processeur, est donné par :

$$\text{Slack} = 1 - U_{\text{périodiques}} = 1 - 0.625 = 0.375$$

Les tâches apériodiques peuvent utiliser ce slack sans perturber l'ordonnancement des tâches périodiques. Le système est ordonnançable si :

$$U_{\text{total}} = U_{\text{périodiques}} + U_{\text{apériodiques}} \leq 1$$

Les tâches apériodiques sont exécutées uniquement pendant ce slack. Ainsi, tant que le taux d'utilisation des tâches apériodiques $U_{\text{apériodiques}} \leq \text{Slack} = 0.375$, le système reste ordonnançable.

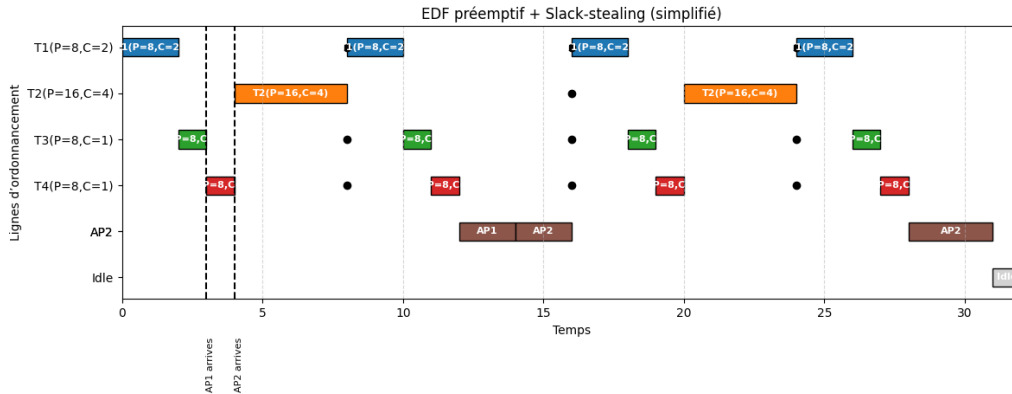


Figure 23: EDF, Slack-Stealing

Le diagramme ci-dessous représente l'ordonnancement avec EDF sans tâche apériodique, illustrant ainsi comment EDF avec Slack-Stealing peut exploiter les périodes d'inactivité (idle) à son avantage.

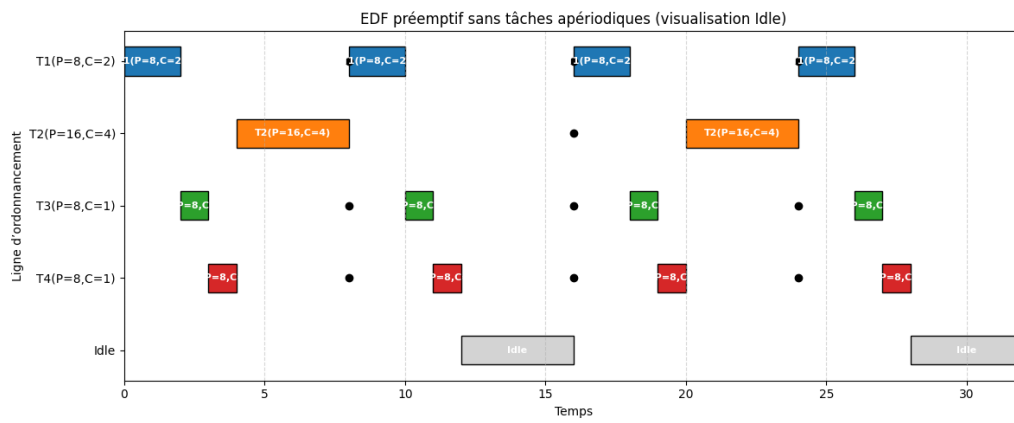


Figure 24: EDF sans tâches apériodiques

Mode 3: Tâches avec des contraintes de précédence

Ensemble 1

$$U_1 = \frac{2}{20} = 0.1, \quad U_2 = \frac{3}{20} = 0.15, \quad U_3 = \frac{4}{20} = 0.2, \quad U_4 = \frac{2}{20} = 0.1$$

La somme des utilisations est :

$$U_{\text{total}} = U_1 + U_2 + U_3 + U_4 = 0.1 + 0.15 + 0.2 + 0.1 = 0.55$$

$$U_{\text{bound}} = 4 \cdot (2^{1/4} - 1) \approx 0.76$$

Les tâches sont **schedulables** avec RM car $U_{\text{total}} = 0.55 < U_{\text{bound}} = 0.76$

Les tâches sont donc **schedulables** avec EDF car $U_{\text{total}} = 0.55 < U_{\text{bound}} \leq 1$

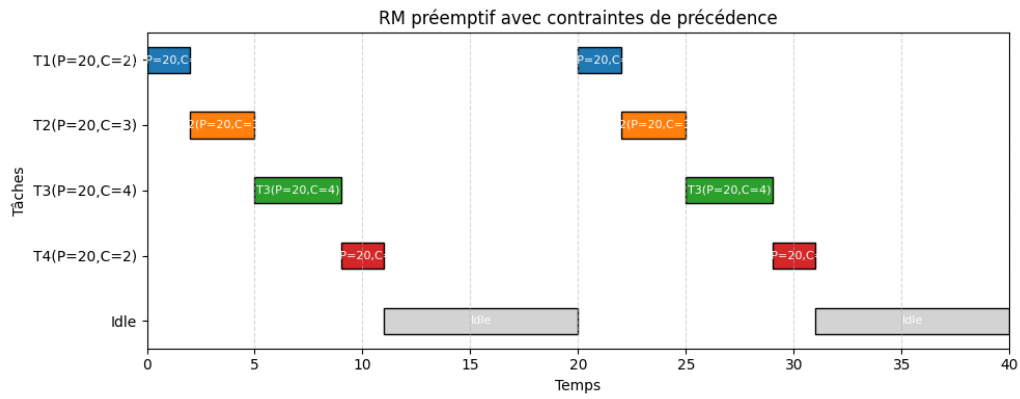


Figure 25: RM, contraintes de précédence, Ensemble 1

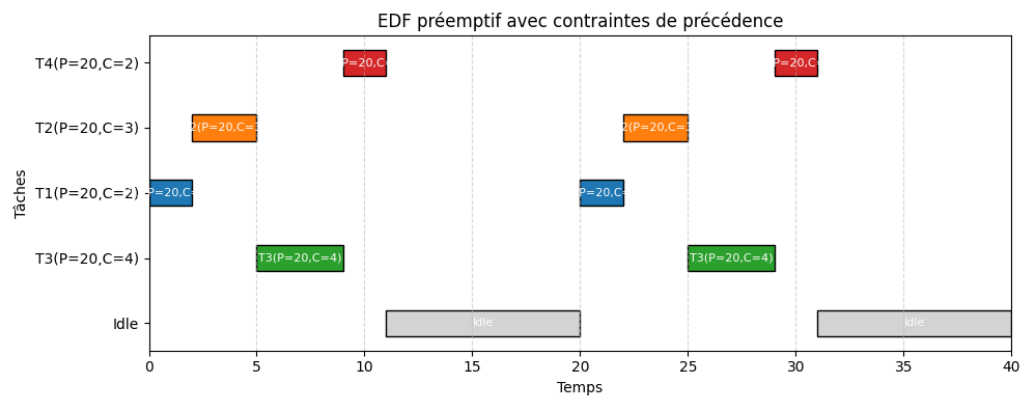


Figure 26: EDF, contraintes de précédence, Ensemble 1

Ensemble 2

$$U_1 = \frac{4}{30} = 0.13, \quad U_2 = \frac{3}{30} = 0.1, \quad U_3 = \frac{4}{30} = 0.13, \quad U_4 = \frac{4}{30} = 0.13$$

La somme des utilisations est :

$$U_{\text{total}} = U_1 + U_2 + U_3 + U_4 = 0.13 + 0.1 + 0.13 + 0.13 = 0.5$$

$$U_{\text{bound}} = 4 \cdot (2^{1/4} - 1) \approx 0.76$$

Les tâches sont **schedulables** avec RM car $U_{\text{total}} = 0.5 < U_{\text{bound}} = 0.76$

Les tâches sont donc **schedulables** avec EDF car $U_{\text{total}} = 0.5 < U_{\text{bound}} \leq 1$

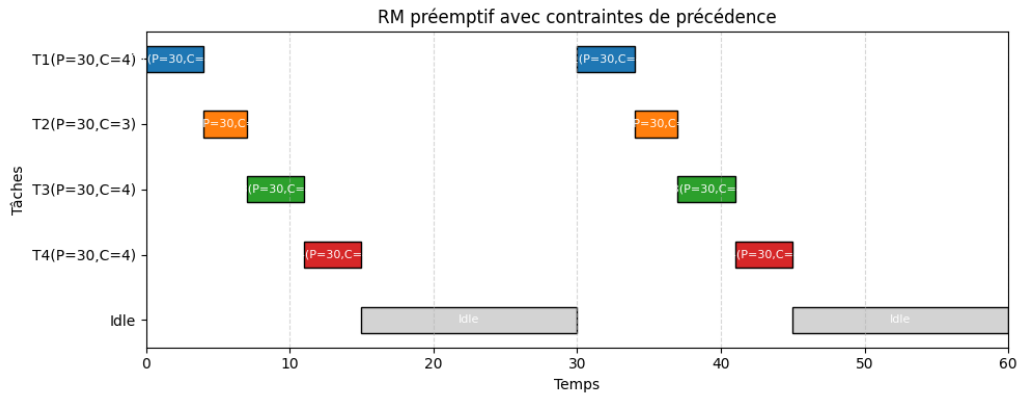


Figure 27: RM, contraintes de précedence, Ensemble 2

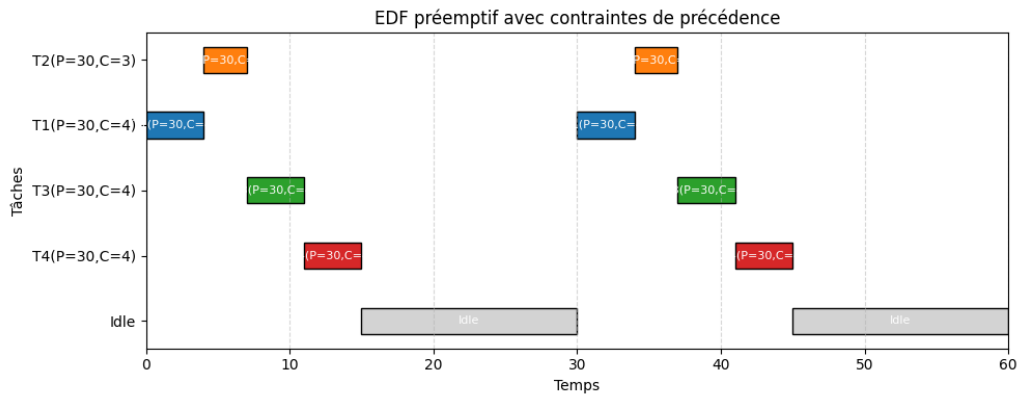


Figure 28: EDF, contraintes de précedence, Ensemble 2

Mode 4: tâches avec contraintes de partage de ressources

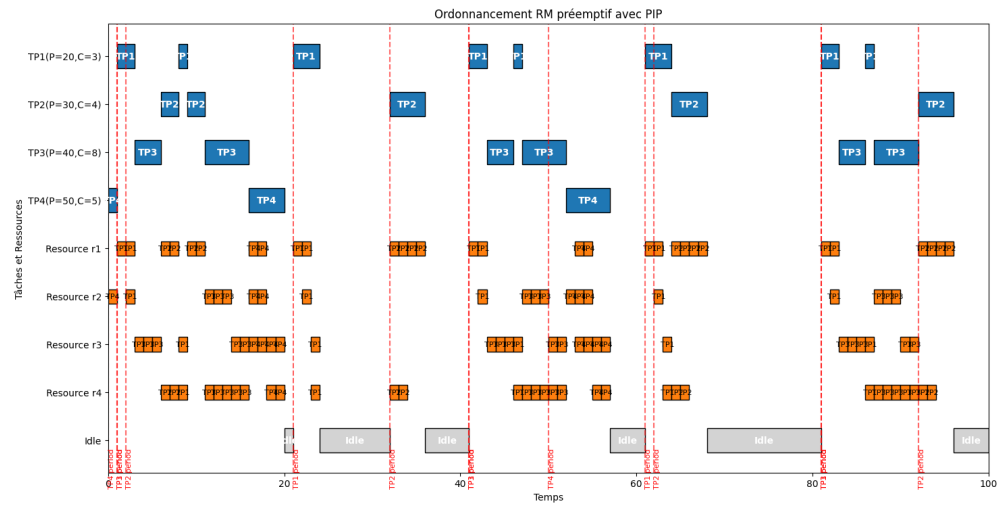


Figure 29: RM - PIP

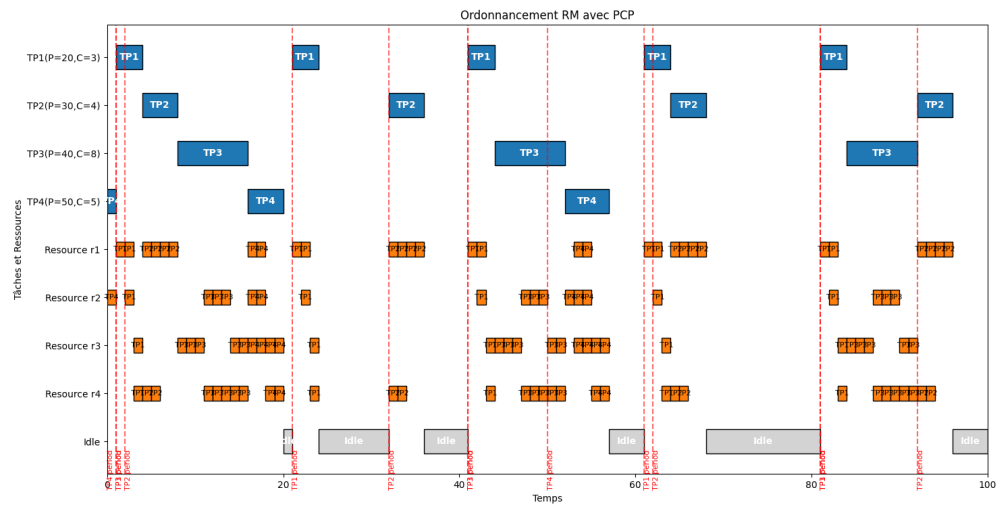


Figure 30: RM - PCP

Étape 3: Analyse d'ordonnabilité assistée

Mode 1: Tâches périodiques indépendantes

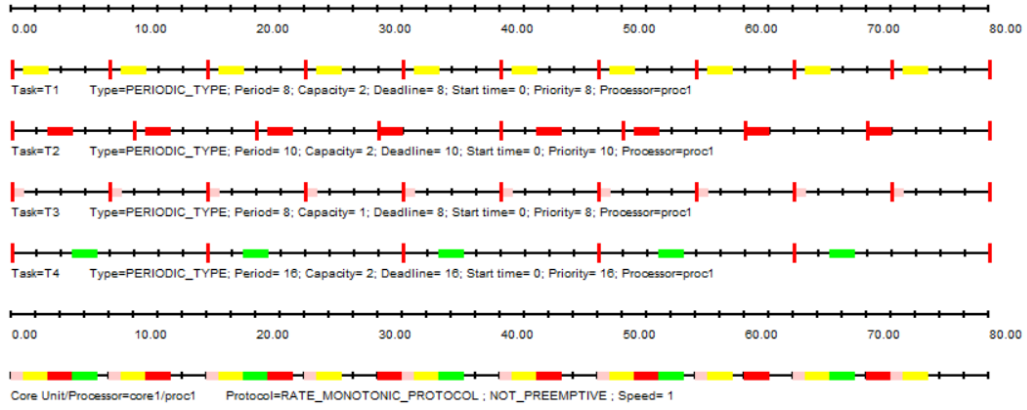


Figure 31: RM Ensemble 1

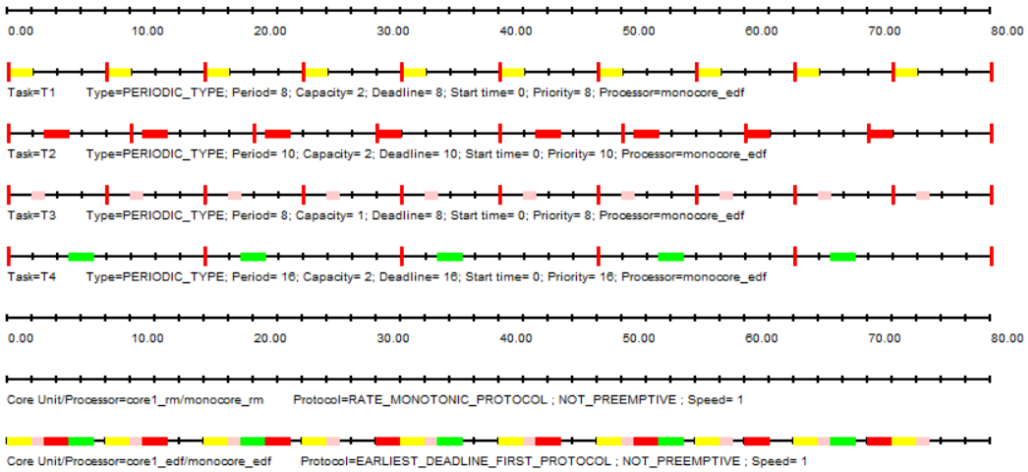


Figure 32: EDF Ensemble 1

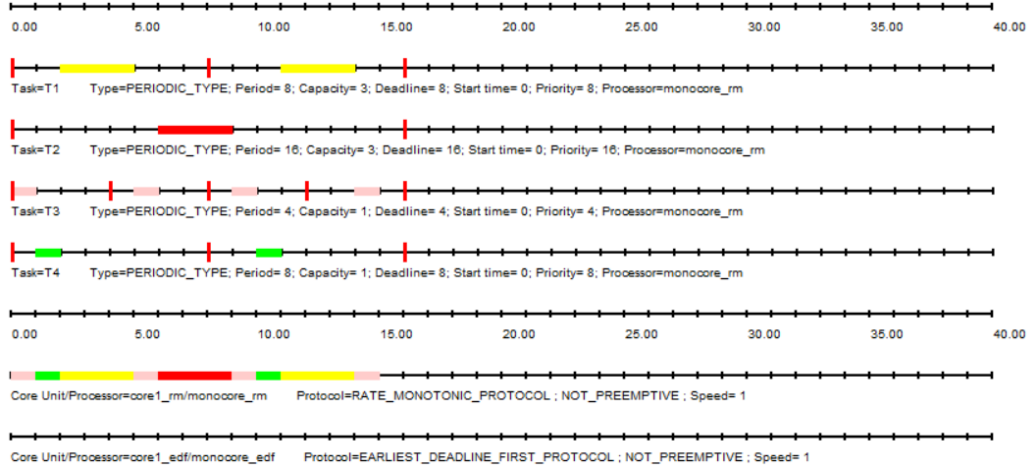


Figure 33: RM Ensemble 2

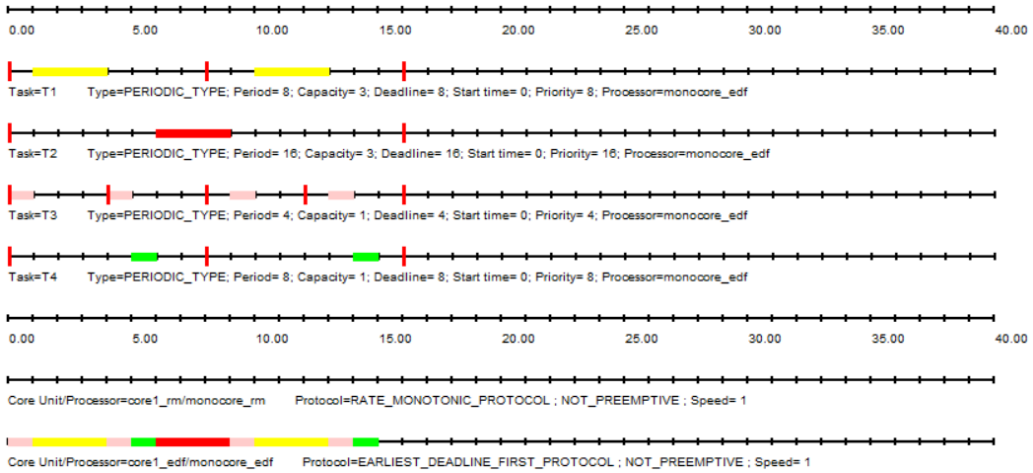


Figure 34: EDF Ensemble 2

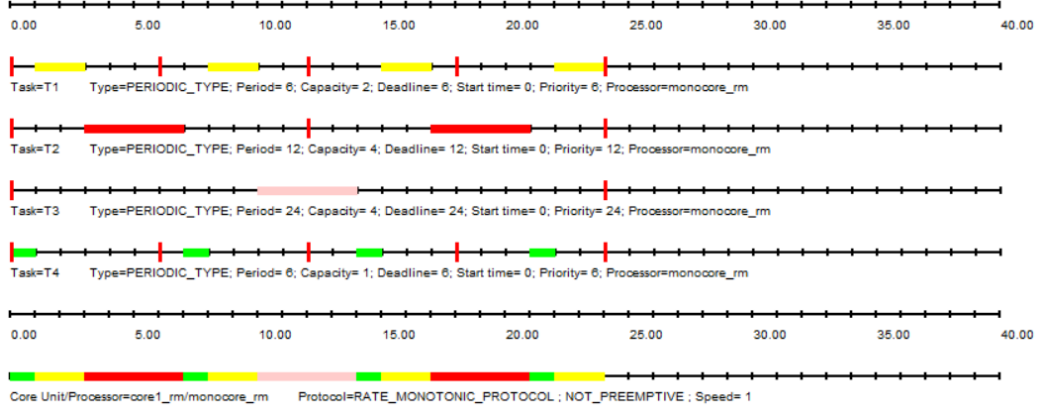


Figure 35: RM Ensemble 4

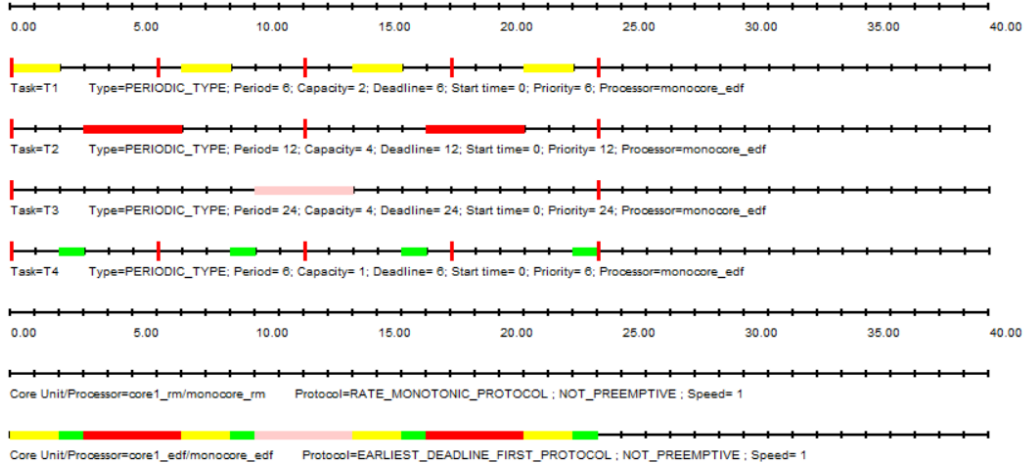


Figure 36: EDF Ensemble 4

Mode 2: Tâches périodiques et tâches aperiodiques indépendantes

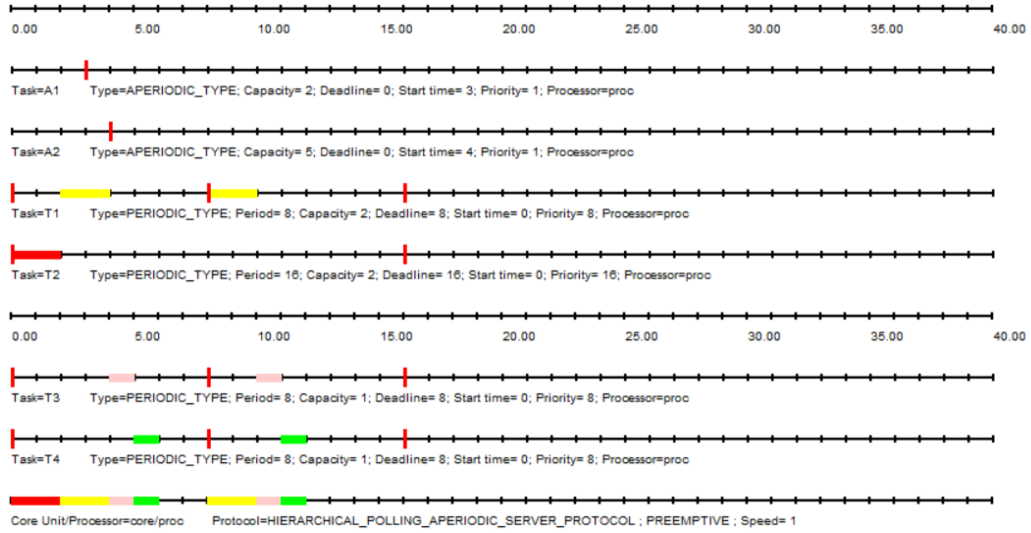


Figure 37: RM Serveur scrutation 1

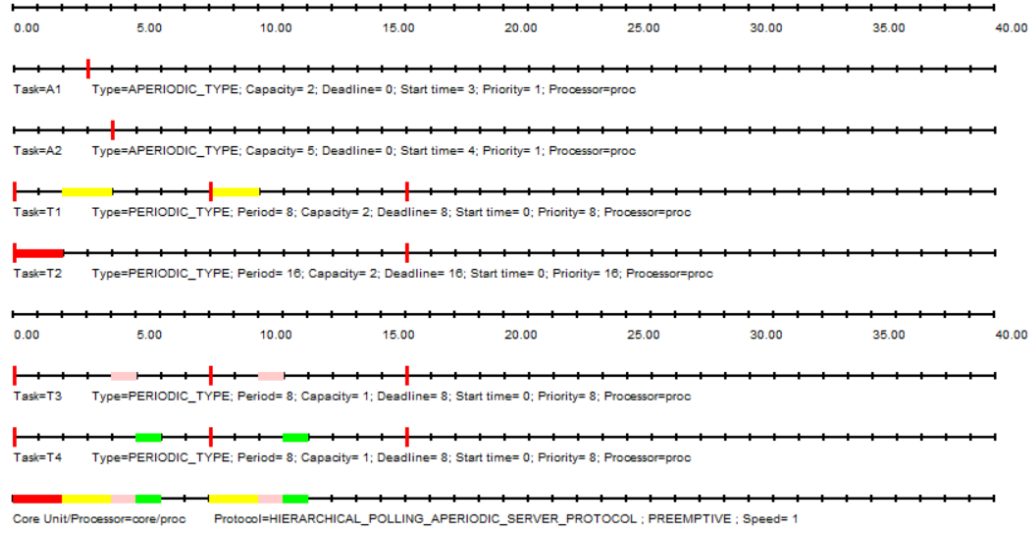


Figure 38: RM Serveur scrutation 2



Figure 39: RM Serveur sporadique 1

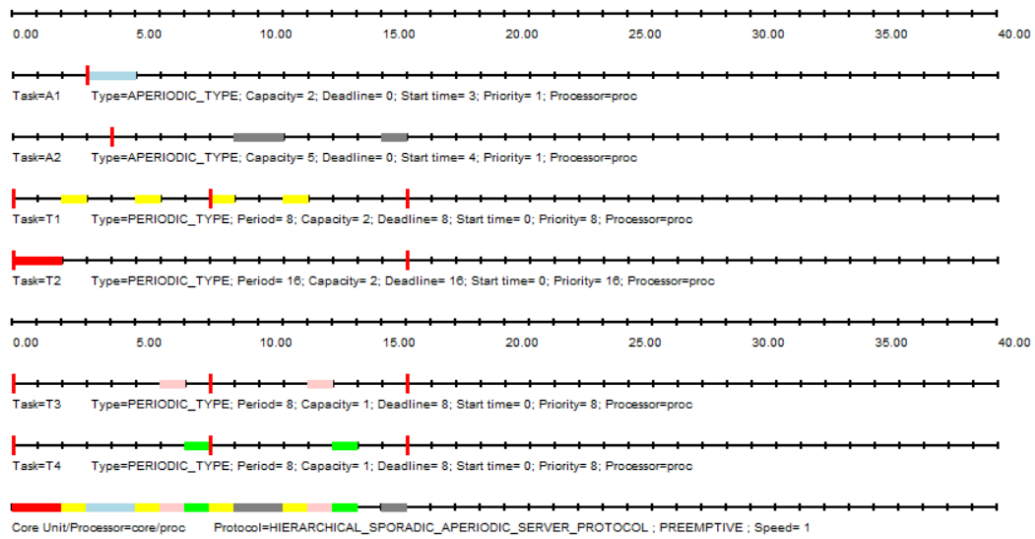


Figure 40: RM Serveur sporadique 2

Mode 3: Tâches avec des contraintes de précedence

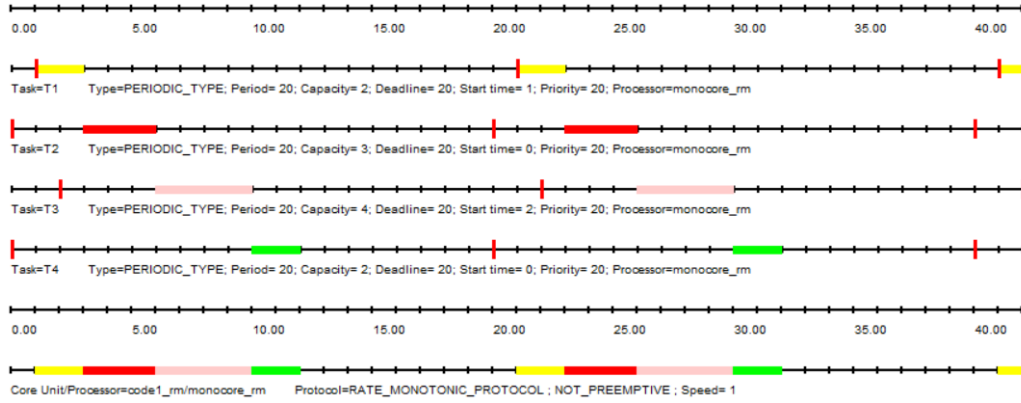


Figure 41: RM Ensemble 1 avec precedence

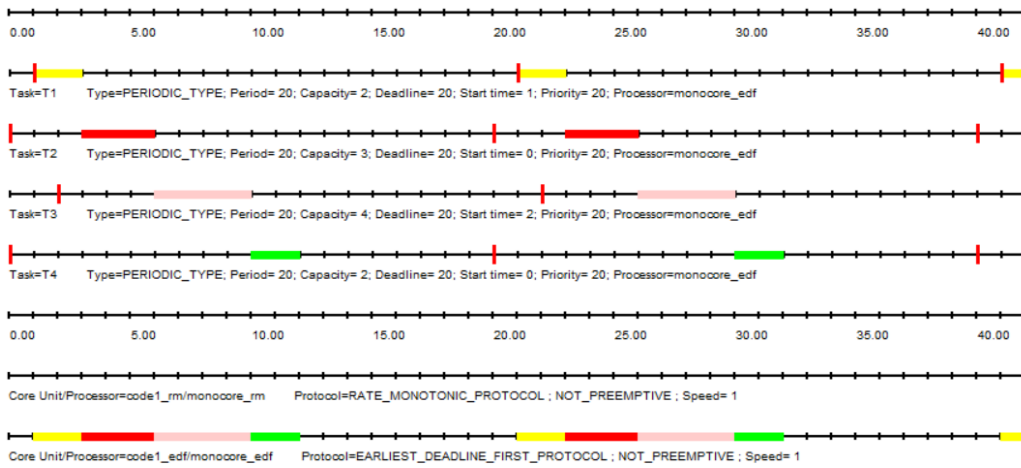


Figure 42: EDF Ensemble 1 avec precedence

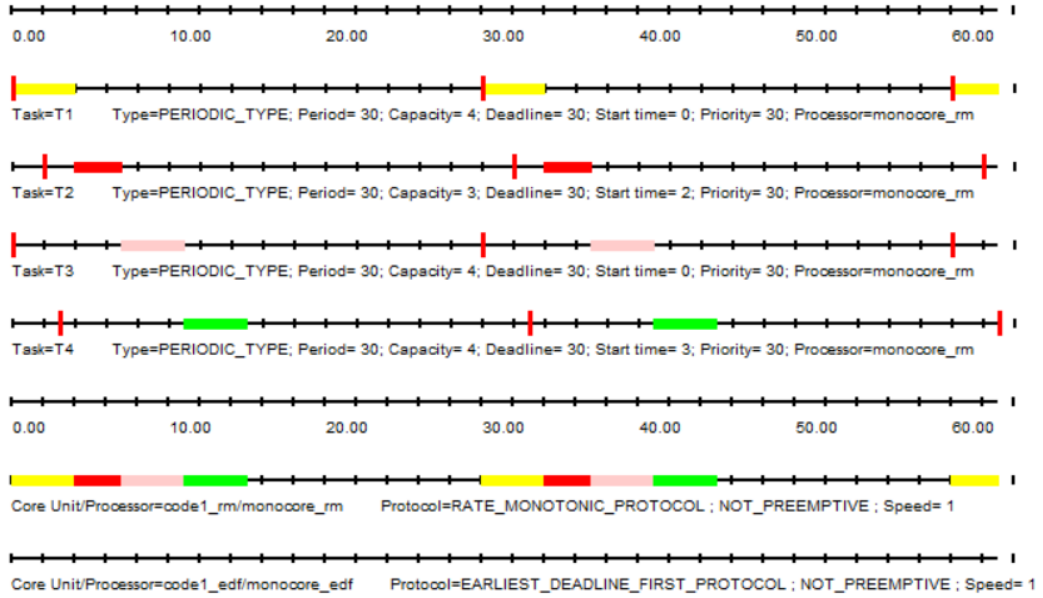


Figure 43: RM Ensemble 2 avec precedence

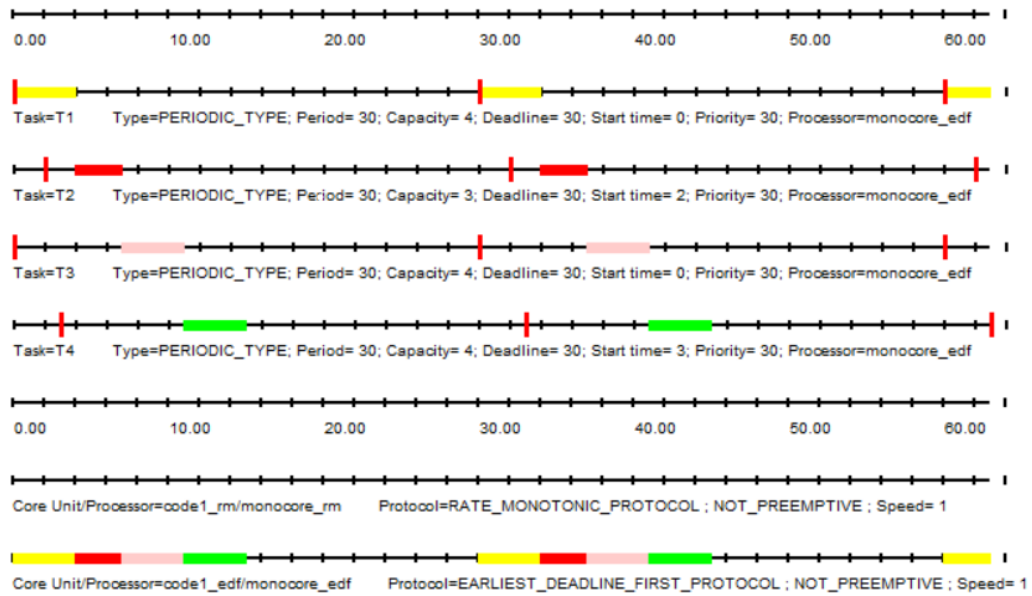


Figure 44: EDF Ensemble 2 avec precedence

Mode 4: tâches avec contraintes de partage de ressources

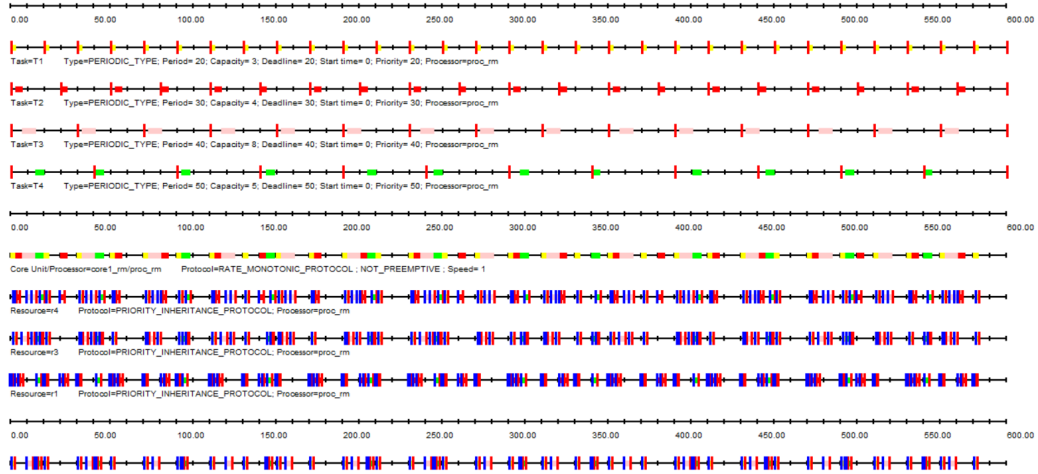


Figure 45: PIP

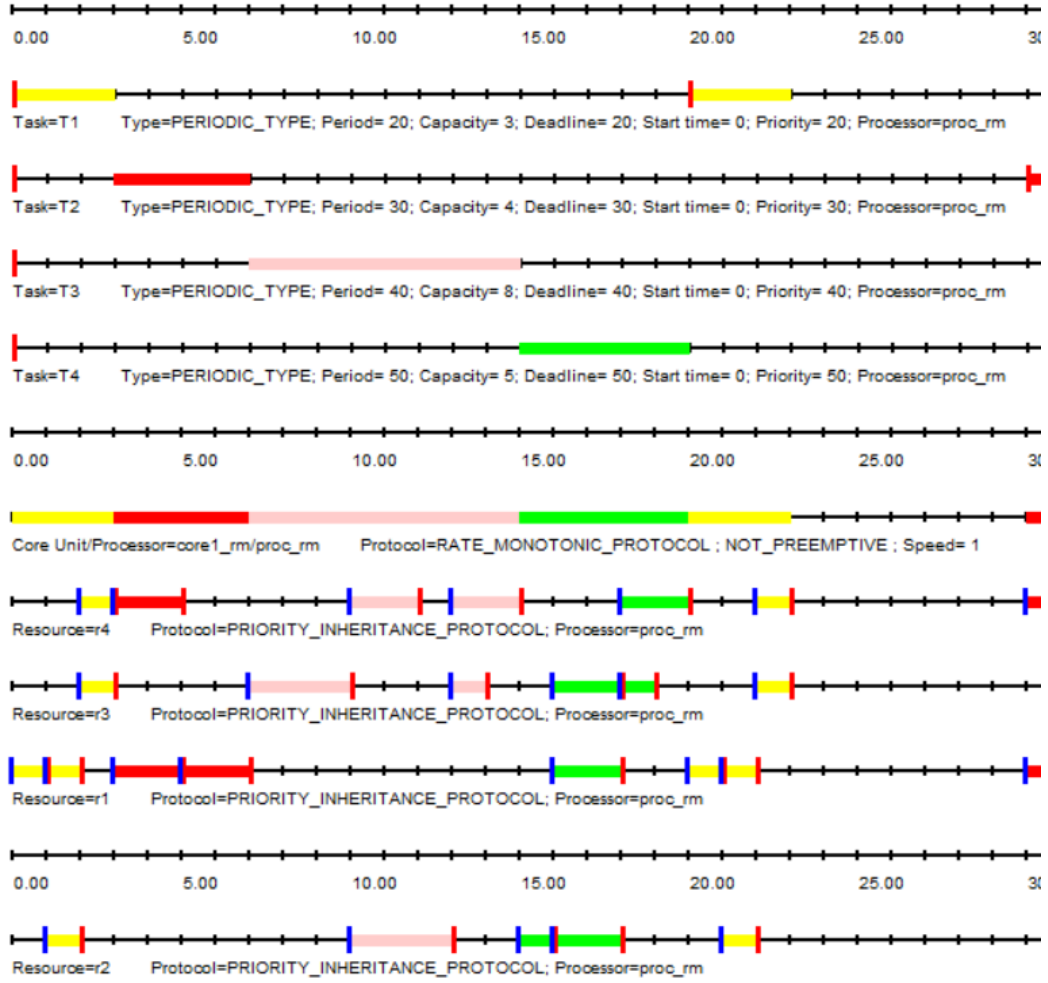


Figure 46: PIP (zoom)

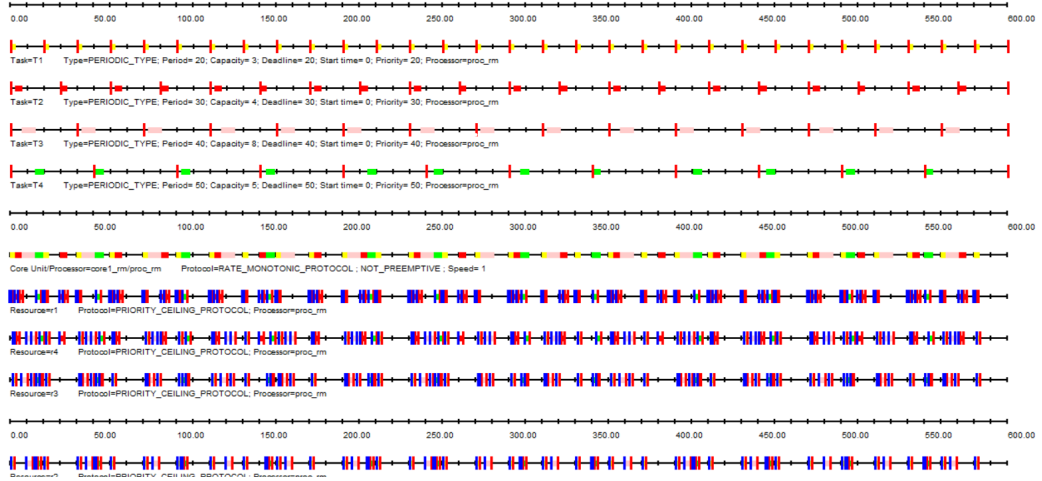


Figure 47: PCP

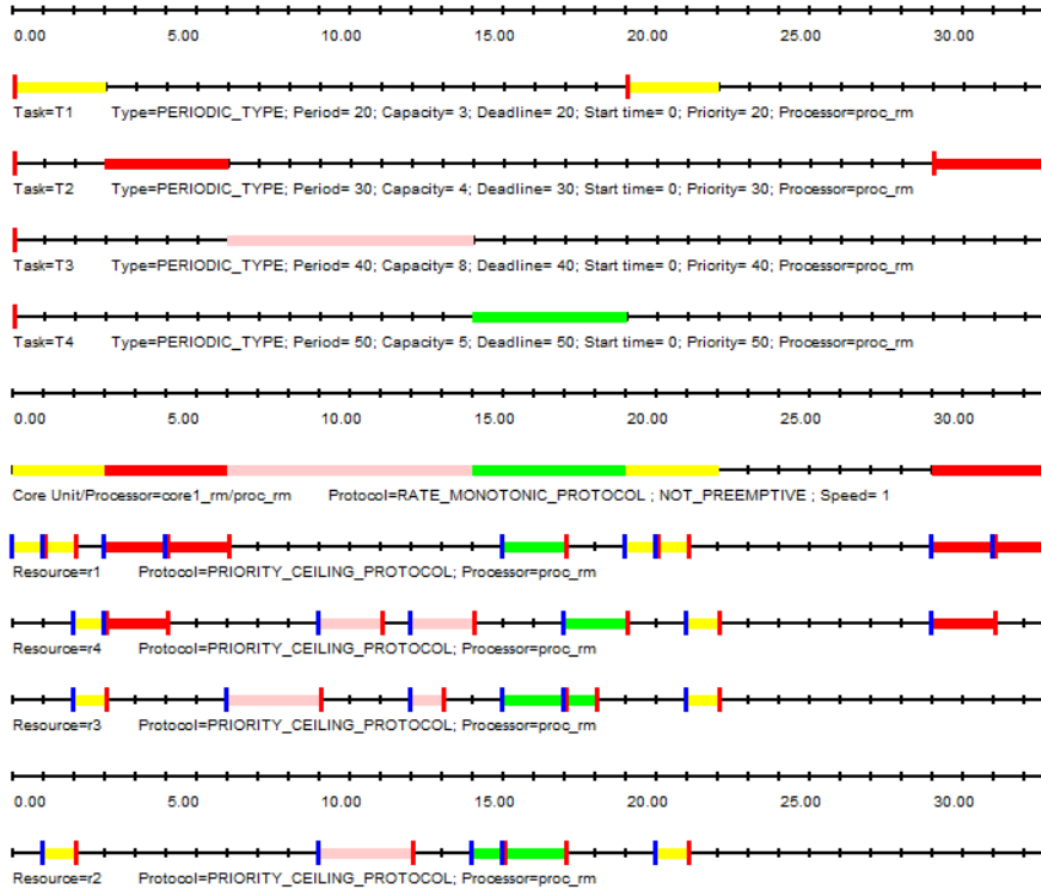


Figure 48: PCP (zoom)